

Smart Expense Categorizer + Spend Clusters

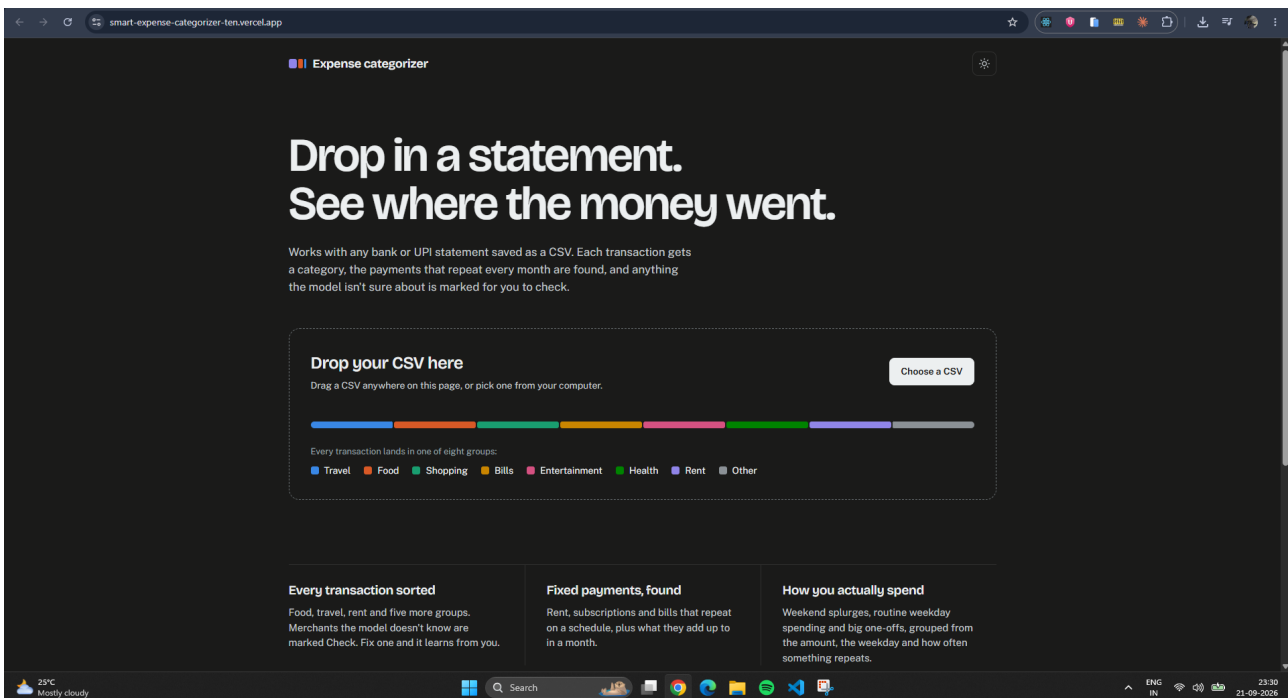
Project report — machine learning over bank and UPI statements · September 2026

1. What the project does

The project turns a raw bank or UPI statement into a readable picture of where money went. A CSV with **date**, **description** and **amount** columns is uploaded; every transaction is labelled into one of eight categories (food, travel, shopping, entertainment, health, bills, rent, other), the payments that repeat on a monthly rhythm are separated out as fixed commitments, and the remaining spending is grouped into behavioural patterns such as weekend splurges, routine weekday spending and big-ticket buys. Where the model is not confident it says so rather than guessing quietly, and a category the user corrects is fed straight back into the model so similar merchants update too.

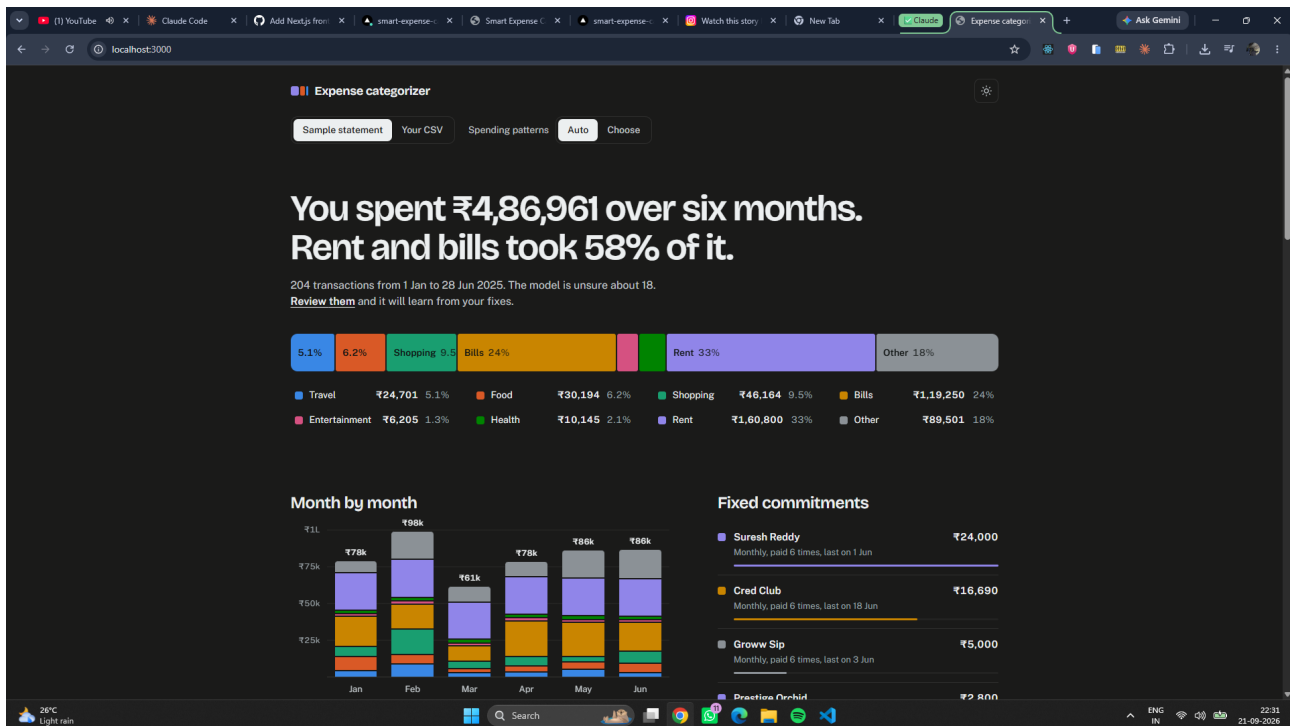
Two front ends sit over the same model code in `src/`: a Next.js web app backed by a FastAPI serverless function, deployed on Vercel, and a Streamlit app for running everything locally in Python. Statements are analysed in memory and never stored.

2. The landing page



The deployed app at `smart-expense-categorizer-ten.vercel.app`. The page starts empty — nothing is pre-loaded — and a CSV can be dropped anywhere on the page or picked from the computer. The colour strip shows the eight categories every transaction lands in.

3. The analysis view



A six-month statement analysed: a sentence-led summary, the whole statement as one proportional strip with per-category totals, month-by-month stacked spend, and the fixed monthly commitments the model found on its own. The 18 rows the model is unsure about are collected behind a single 'Review them' link.

The read is immediate: ₹4,86,961 over 204 transactions from 1 January to 28 June 2025, with rent (33%) and bills (24%) together taking 58% of everything spent. February is the heaviest month at ₹98k against a ₹78–86k norm. Under fixed commitments the model has picked out a ₹24,000 monthly rent to Suresh Reddy, a ₹16,690 Cred Club bill, a ₹5,000 Groww SIP and a ₹2,800 society charge — each paid six times, recognised from the rhythm of the payments alone rather than from any list of known billers. Figures use Indian digit grouping, and both a light and a dark theme are available.

4. How it works

Categorizer (`src/categorizer.py`). Descriptions are lowercased and stripped of reference numbers, so UPI/SWIGGY/ORDER482913 becomes `upi swiggy order`. Word 1–2 grams carry the merchant name and character 2–5 grams catch spelling drift (NETFLIX COM, PG*ZOMATO*). A logistic regression, a ComplementNB and a calibrated linear SVM vote on the label; the averaged probability is the confidence, and anything below 0.5 is flagged for a human check. Training deliberately pairs invented brand names with generic descriptors (... PHARMACY, ... PETROL PUMP) so an unknown brand with a telling word still lands correctly.

Recurring payments (`src/clusters.py`). A merchant counts as recurring when it is paid on a steady monthly rhythm (three or more payments, gaps near a month and not erratic, which catches variable bills like electricity) or repeats at a fixed amount at least three weeks apart. Frequency alone is not enough — restaurants and petrol pumps recur too, and they are not commitments.

Patterns. Each transaction becomes six features: log amount, weekend flag, a circular day-of-month pair, an hour bucket and a recurring flag. These are standard-scaled and clustered with K-Means, with k chosen by silhouette score (3 to 6 on a full statement, capped at about one pattern per ten transactions). Each cluster is named from its own statistics.

5. Results

The training data is synthetic — real brand names with random amounts, dates and reference numbers across several statement formats. The numbers therefore measure how well the model handles brands and formats it has never seen, not how it will do on one particular bank's export.

Measure	Before	Now
Accuracy on a 204-row held-out statement, unseen brands	0.544	0.877 (macro-F1 0.904)
Merchant-grouped cross-validation (whole merchants held out)	not measured	0.662
Share of mistakes carrying a low-confidence flag	none flagged	64%, flagging only 5% of correct rows
Cluster separation (silhouette) on the held-out statement	~0.07	0.18
Accuracy after fixing one row per misread brand (6 fixes)	—	1.000

Two caveats are worth stating plainly. The held-out statement was used to choose among the top few ensemble variants, so 0.877 is slightly optimistic; and the remaining mistakes are all wholly unfamiliar brand names with no descriptor word (Rapido, Dunzo, Cleartrip), which nothing in the text can fix — the low-confidence flag and the user's own corrections are the answer there. Random train/test splits flatter a text classifier because the same merchant lands on both sides of the split, which is why merchant-grouped validation is reported instead.

6. Architecture and layout

Path	Role
app/, components/, lib/	Next.js App Router page, spend strip, month chart, commitments, patterns, transactions table, shared types and Indian-format helpers
api/index.py	FastAPI function behind POST /api/analyze; fits the classifier from the seed data in about half a second and caches it for warm instances
src/categorizer.py	TF-IDF word + character features, voting ensemble, grouped CV, scoring
src/clusters.py	Date parsing, merchant identity, recurring detection, K-Means and cluster naming
data/	Deterministic generator, ~1,400-row labelled seed set, six-month holdout statement with answer key
app.py, train.py	Streamlit UI and the CLI trainer/scorer
tests/	pytest suite over the model, clustering and API

The API accepts the CSV file plus an optional `n_clusters` (0 chooses automatically) and a JSON list of corrections, and returns category totals, monthly totals, recurring payments, pattern summaries and every categorized row. Bad input comes back as a 400 with a readable message — a missing column, an empty file, or a non-numeric amount reported with its CSV line number. Stack: Next.js 16, React 19, TypeScript and Tailwind 4 on the front end; pandas, scikit-learn and FastAPI behind it.

7. Where it stands

The web app is deployed and working end to end on Vercel, the model comfortably beats the baseline it replaced on the honest measure (0.544 to 0.877 on unseen brands), the uncertainty flag catches roughly two thirds of mistakes while disturbing only one row in twenty that was already right, and a handful of user corrections takes the held-out statement to perfect. The clearest next step is training on real labelled statements rather than synthetic ones, since merchant strings differ from bank to bank — the CLI trainer already supports this through `python train.py --data data/my_labeled.csv`.